

Problem Set Solutions: Seminar 05

Heaps I
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: Binary heaps, priority queues, Huffman coding, heap simulations
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. A heap is the canonical data structure for keeping only the currently relevant extreme element while the underlying set changes. The same idea appears in many forms: k -way merge stores only the current heads of sorted lists; Huffman coding repeatedly merges the two smallest weights; kinetic simulations process the next collision event; online medians maintain two complementary priority queues. The important invariant is not the tree shape itself, but the fact that a local frontier is enough to identify the next globally forced step.

Contents

Problem 1. k -way merge	2
Problem 2. Heapsort without reversing the final array	2
Problem 3. Lazy deletion when deleting absent elements is allowed	3
Problem 4. Constructing a Huffman code in $O(k \log k)$	4
Problem 5. Merging moving groups on a line	4
Problem 6. Converting a min-heap array into a max-heap in $O(n)$	6
Problem 7. The smallest k elements of a min-heap	6
Problem 8. Online median after every insertion	7
Problem 9. Building a heap by calling <code>siftUp</code> from n down to 1	8
Problem 10. Sorting a k -displaced sorted array	8
Problem 11. Repeatedly removing the closest pair	9
Problem 12. Priority queue functionality using sorted decreasing arrays	10

Conventions. Arrays are indexed from 1 unless stated otherwise. A binary heap stored in an array uses children $2i$ and $2i + 1$. A min-heap returns the smallest key; a max-heap returns the largest key. For priority queues with lazy deletion, equal keys are handled by multiplicities.

Problem 1. k -way merge

Given k arrays $A_1, A_2, \dots, A_k$ sorted in increasing order, merge them into one sorted array in time

$$O((|A_1| + \dots + |A_k|) \log k).$$

Solution.

Idea. At any moment, the smallest not-yet-output element of each array is its current head. Therefore the next global output element is the minimum among at most k heads.

Algorithm. For every nonempty array A_t , insert the triple

$$(A_t[1], t, 1)$$

into a min-priority queue, where the fields are value, array index, and position inside that array. Then repeat until the priority queue is empty:

1. Extract the triple (x, t, p) with minimum value x .
2. Append x to the answer.
3. If $p < |A_t|$, insert $(A_t[p + 1], t, p + 1)$ into the priority queue.

Correctness. For each array A_t , the heap contains exactly its first not-yet-output element, unless the whole array has already been exhausted. Since each A_t is sorted, every later element of A_t is at least this current head. Hence the smallest element among all not-yet-output elements over all arrays must be one of the heads stored in the heap. Extracting the minimum heap element therefore outputs the next element of the globally sorted merge. After outputting an element from A_t , inserting its successor restores the invariant. By induction over the number of output elements, the algorithm outputs all elements in nondecreasing order.

Running time. Let $N = |A_1| + \dots + |A_k|$. Each element is inserted into and extracted from a heap of size at most k once. Therefore the running time is $O(N \log k)$ and the extra memory is $O(k)$.

Problem 2. Heapsort without reversing the final array

Implement heapsort so that the final reversal of the array is unnecessary.

Solution.

Idea. If we repeatedly extract the minimum into a separate output array, the output is increasing. If we instead sort in-place and want the final array increasing, we should use a max-heap: every extraction moves the current maximum to its final position at the right end.

Algorithm. Build a max-heap on the whole array. For $r = n, n-1, \dots, 2$:

1. Swap the heap root a_1 , the current maximum, with a_r .
2. Decrease the heap size to $r-1$.
3. Run SIFTDOWN from the root using the max-heap comparator.

After the loop, the array is sorted increasingly from left to right.

Correctness. After the max-heap is built, the largest element of the current heap is at the root. At iteration r , the suffix $a_{r+1}, \dots, a_n$ already contains the largest $n-r$ elements in sorted order. Swapping the root with a_r places the largest remaining element exactly at position r , which is its final sorted position. The subsequent SIFTDOWN restores the max-heap property on the remaining prefix. By induction over decreasing r , after the last iteration every position contains the correct element in increasing order. No reversal is needed because maxima were written from right to left.

Running time. Building the heap by Floyd's algorithm takes $O(n)$ time. There are $n-1$ extractions, each costing $O(\log n)$. Total time is $O(n \log n)$ and the extra memory is $O(1)$.

Problem 3. Lazy deletion when deleting absent elements is allowed

Recall the lazy deletion method with two heaps: inserted elements are stored in A , deleted elements in D , and equal roots are removed from both heaps before answering GETMIN or EXTRACTMIN. This assumes that only previously inserted elements may be deleted. How should we handle ERASE(x) if x may be absent?

Solution.

Idea. The dangerous operation is scheduling a deletion of an element that is not currently present: a future insertion of the same key could be incorrectly deleted. Therefore deletion requests must be filtered by live multiplicities.

Algorithm. Maintain a hash table $c[x]$, the number of currently live copies of x . There are two equivalent implementations.

Version 1: one heap plus multiplicities. Use one min-heap A containing all inserted copies, including stale ones.

- INSERT(x): push x into A and set $c[x] \leftarrow c[x] + 1$.
- ERASE(x): if $c[x] > 0$, set $c[x] \leftarrow c[x] - 1$; otherwise ignore the request.
- Before GETMIN or EXTRACTMIN, while A is nonempty and $c[\min A] = 0$, pop the root from A .
- GETMIN: after cleaning, return the root.
- EXTRACTMIN: after cleaning, pop the root x and set $c[x] \leftarrow c[x] - 1$.

Version 2: two heaps plus multiplicities. Keep the original heap D of scheduled deletions, but push x into D only if $c[x] > 0$, and simultaneously decrement $c[x]$. If $c[x] = 0$, ignore ERASE(x). Then the usual cleaning step, popping equal roots from A and D , is safe.

Correctness. The table c is the exact multiplicity of live elements. A deletion request for an absent x has $c[x] = 0$, so ignoring it preserves the represented multiset. A deletion request for a present x decreases $c[x]$ by one, which removes exactly one live copy. In the one-heap version, a heap root with $c[x] = 0$ represents only stale deleted copies and can be removed without changing the live multiset. Once the root has positive multiplicity, no smaller live key exists; otherwise it would be above or before this root after all stale smaller keys were removed. Therefore GETMIN and EXTRACTMIN return the correct minimum.

Running time. Every inserted heap copy is physically popped at most once. Thus over q operations the total number of heap pops and pushes is $O(q)$, and each costs $O(\log q)$. The amortized time per operation is $O(\log q)$ and the memory is $O(q)$.

Problem 4. Constructing a Huffman code in $O(k \log k)$

An alphabet has k symbols, and symbol i appears n_i times. Build the Huffman code by repeatedly merging the two least frequent symbols or subtrees.

Solution.

Idea. The recursive definition says that the two least frequent symbols should become siblings at maximum depth. A min-heap gives exactly the next two weights that must be merged.

Algorithm. Create a leaf node for every symbol i with weight n_i . Insert all leaves into a min-priority queue keyed by weight. While the queue contains more than one node:

1. Extract the two nodes u and v with smallest weights.
2. Create a new internal node w with children u, v and weight $\text{weight}(u) + \text{weight}(v)$.
3. Insert w back into the priority queue.

The remaining node is the root of the Huffman tree. Assign bit 0 to one edge from every internal node and bit 1 to the other. The codeword of a symbol is the sequence of edge labels on the path from the root to its leaf.

Correctness. At each step the algorithm performs exactly the recursive Huffman operation: it selects the two smallest currently available frequencies and replaces them by their sum. This reduces the number of active items by one while preserving the contribution of the two merged items: after the final code is expanded, the two children receive the same prefix and then one additional bit each. Thus the constructed tree is exactly the tree defined by the Huffman recurrence. The final root contains all k leaves, and each leaf path gives a prefix-free binary codeword.

Running time. The priority queue initially contains k items. It performs $k - 1$ merge iterations, each with two extracts and one insert, all on a heap of size at most k . The total running time is $O(k \log k)$. The tree has $2k - 1$ nodes, so the memory is $O(k)$.

Problem 5. Merging moving groups on a line

Several groups of people stand on a coordinate line. A group of size s moves to the right with speed $1/s$. When one group catches another, the two groups merge, their sizes add, and the new group changes speed. Simulate all mergers and report the sizes of all final groups in $O(n \log n)$ time.

Solution.

Idea. Only adjacent groups can collide first: a group cannot reach a non-adjacent group without first meeting every group between them. Therefore the process is an event simulation over adjacent pairs. The next event is the adjacent collision with smallest time.

Algorithm. First sort the groups by initial coordinate. For each current group g , store:

$$(s_g, x_g, t_g, v_g), \quad v_g = 1/s_g,$$

where x_g is the position of the group at time t_g . Its position at time $t \geq t_g$ is

$$\text{pos}(g, t) = x_g + v_g(t - t_g).$$

Maintain the current groups in a doubly linked list. For every adjacent pair (L, R) , compute its collision time. At current time T , let

$$d = \text{pos}(R, T) - \text{pos}(L, T).$$

If $v_L \leq v_R$, the left group never catches the right one, so no event is inserted. If $v_L > v_R$, insert the event

$$T + \frac{d}{v_L - v_R}$$

into a min-priority queue, together with the identities or version numbers of L and R .

Repeatedly extract the earliest event. If its two groups are no longer alive, are no longer adjacent, or their version numbers changed, discard it as stale. Otherwise merge them at the event time T and collision position X . The new group has

$$s = s_L + s_R, \quad v = 1/s, \quad x = X, \quad t = T.$$

Replace L, R by this group in the linked list and insert possible new events with its new left and right neighbors. When the priority queue becomes empty, output the sizes in linked-list order.

Correctness. Any first collision in the system must occur between adjacent current groups. If two non-adjacent groups met first, one of them would have to pass through an intermediate group earlier, which is impossible without a prior collision. Hence it is sufficient to schedule only adjacent-pair events. The priority queue extracts the earliest scheduled adjacent collision. Stale events are ignored, so every processed event corresponds to two groups that are still adjacent and whose computed collision time is still valid. Replacing these two groups by their merged group exactly matches the physical rule of the process. By induction over processed events, after each merge the linked list and priority queue describe precisely all possible next adjacent collisions of the real system. When the queue is empty, no adjacent left group is faster than its right neighbor, so no future collision is possible. The remaining linked-list groups are exactly the final groups.

Running time. There are exactly $n - m$ processed mergers, where m is the final number of groups, so at most $n - 1$ real events. Each real event creates only $O(1)$ new candidate events. Thus $O(n)$ events are inserted, plus stale copies. Every priority-queue operation costs $O(\log n)$. Sorting costs $O(n \log n)$, and the simulation costs $O(n \log n)$; memory is $O(n)$.

Problem 6. Converting a min-heap array into a max-heap in $O(n)$

A binary min-heap on n elements is stored as an array. Rebuild the array into a binary max-heap in $O(n)$ time.

Solution.

Idea. The existing min-heap order is not useful for max-heap order, but the array already contains all elements. Floyd's bottom-up heap construction builds any heap order in linear time.

Algorithm. For $i = \lfloor n/2 \rfloor, \lfloor n/2 \rfloor - 1, \dots, 1$, run `SIFTDOWNMAX`(i), comparing each node with its larger child and swapping while the child is larger.

Correctness. Process vertices in reverse level order. When index i is processed, both subtrees rooted at its children are already max-heaps. The operation `SIFTDOWNMAX` moves a_i down until it is at least as large as its children, while preserving the heap property inside the child subtrees. Hence after processing i , the subtree rooted at i is a max-heap. By induction upward from the leaves, after processing $i = 1$ the whole array is a max-heap.

Running time. The total cost of bottom-up heap construction is

$$\sum_{h \geq 0} O\left(\frac{n}{2^{h+1}} h\right) = O(n).$$

The algorithm is in-place and uses $O(1)$ extra memory.

Problem 7. The smallest k elements of a min-heap

A binary min-heap on n elements is stored as an array. Given k , find the smallest k elements in time $O(k \log k)$.

Solution.

Idea. The children of an extracted heap node are the only new candidates that can become globally small. We should not scan the whole heap; we maintain a priority queue of the frontier of the heap tree.

Algorithm. Create an auxiliary min-priority queue Q of heap indices, keyed by their array values. Insert index 1. Repeat k times, or until the original heap is exhausted:

1. Extract an index i of minimum key a_i from Q and output a_i .
2. If $2i \leq n$, insert $2i$ into Q .
3. If $2i + 1 \leq n$, insert $2i + 1$ into Q .

Correctness. After some outputs, call an unreported node exposed if its parent has already been reported, or if it is the root. The auxiliary queue contains exactly the exposed unreported nodes. Every unreported non-exposed node has an unreported ancestor above it. By the min-heap property, that ancestor's key is no larger than the node's key. Therefore the smallest unreported element must be exposed and hence is present in Q . Extracting the minimum from Q outputs the next smallest original heap element. Inserting its children restores the exposed-frontier invariant. By induction, the algorithm outputs the k smallest elements.

Running time. At most k indices are extracted and at most $2k$ child indices are inserted. The auxiliary heap size is $O(k)$, so the total time is $O(k \log k)$ and the extra memory is $O(k)$.

Problem 8. Online median after every insertion

Integers are inserted online into a multiset. After every insertion, report the current median. Each request must be processed in $O(\log n)$ time.

Solution.

Idea. Keep the smaller half in a max-heap and the larger half in a min-heap. The median lies at the boundary between the two halves.

Algorithm. Maintain:

- L : a max-heap containing the lower half of the elements;
- R : a min-heap containing the upper half of the elements.

The invariants are

$$|L| = |R| \quad \text{or} \quad |L| = |R| + 1, \quad \max L \leq \min R$$

whenever both heaps are nonempty.

To insert x , compare it with $\max L$ if L is nonempty. Insert x into L if L is empty or $x \leq \max L$, otherwise insert it into R . Then rebalance: if $|L| > |R| + 1$, move $\max L$ to R ; if $|R| > |L|$, move $\min R$ to L .

Report $\max L$ as the lower median. If the convention requires the arithmetic median for an even number of elements, report

$$\frac{\max L + \min R}{2}$$

when $|L| = |R|$, and $\max L$ otherwise.

Correctness. The ordering invariant says every element of L is at most every element of R . The size invariant says L contains exactly the first $\lceil m/2 \rceil$ elements of the sorted multiset of size m . Therefore its maximum is the lower median. If arithmetic medians are required for even m , the two middle elements are exactly $\max L$ and $\min R$. The insertion step may violate only the size invariant by one element; moving the boundary element between heaps restores both the size invariant and the ordering invariant. Hence the reported value after every update is correct.

Running time. Each insertion performs a constant number of heap insertions/extractions, each in $O(\log n)$ time. The memory is $O(n)$.

Problem 9. Building a heap by calling `siftUp` from n down to 1

Consider the following procedure: for every element from the n -th down to the first, call `SIFTUP`. Is the resulting binary min-heap always correct?

Solution.

Answer. No.

Idea. A later `SIFTUP` call can move a large element down into a subtree that was already processed. Since that subtree will not be repaired again, the final array can violate the heap property.

Algorithm. A counterexample with four distinct elements is

$$[3, 1, 4, 2].$$

Run `SIFTUP` from positions 4, 3, 2, 1.

- At position 4, the value 2 has parent 1, so nothing changes.
- At position 3, the value 4 has parent 3, so nothing changes.
- At position 2, the value 1 has parent 3, so they are swapped. The array becomes

$$[1, 3, 4, 2].$$

- Position 1 needs no action.

The final array is not a min-heap, because position 2 has value 3 and its child position 4 has value 2, so $3 > 2$.

Correctness. The displayed execution follows the described procedure exactly and ends in an array violating the heap inequality. Therefore the procedure is not guaranteed to build a valid heap.

Running time. The claim is refuted by a constant-size counterexample; no asymptotic bound is needed for the disproof.

Problem 10. Sorting a k -displaced sorted array

Every element of a sorted array of length n was moved by at most k positions. Sort the array in time $O(n \log k)$.

Solution.

Idea. In the final sorted order, the next output element cannot be farther than k positions ahead in the current array. Thus the next minimum is always among a sliding window of $k + 1$ candidates.

Algorithm. Insert the first $k+1$ elements of the array into a min-heap. Then scan the remaining elements from left to right. At each step:

1. Extract the minimum from the heap and append it to the output.
2. Insert the next array element into the heap.

After all input elements have been inserted, repeatedly extract the remaining heap elements and append them to the output.

Correctness. Consider the element that should appear next in sorted order at output position p . Since every element moved by at most k positions from its sorted position, this element appears in the input no later than position $p+k$. At the moment when position p is produced, all not-yet-output elements from the input prefix up to $p+k$ have been inserted into the heap, and no smaller element can lie beyond that prefix. Therefore the heap minimum is exactly the next sorted element. By induction over p , the algorithm outputs the whole array in sorted order.

Running time. The heap size is at most $k+1$. Each of the n elements is inserted once and extracted once, so the total running time is $O(n \log k)$ and the extra memory is $O(k)$.

Problem 11. Repeatedly removing the closest pair

An array contains $2n$ numbers. At each step, choose two numbers with minimum absolute difference; if there are several such pairs, choose a pair with maximum sum. Remove the chosen pair. Determine the order of removals in $O(n \log n)$ time.

Solution.

Idea. In a sorted set of remaining numbers, a closest pair is always adjacent. After removing an adjacent pair, the only new adjacent pair that can appear is the pair formed by its two former outer neighbors. This is exactly a heap plus doubly linked list problem.

Algorithm. Sort the numbers increasingly, keeping original indices if the output must identify original elements. Put the sorted elements into a doubly linked list. For every adjacent pair (u, v) with values $x_u \leq x_v$, insert the candidate into a priority queue with key

$$(x_v - x_u, -(x_u + x_v)).$$

Thus the priority queue first minimizes difference and then maximizes sum.

Repeat n times:

1. Extract the best candidate pair (u, v) .
2. If u or v has already been deleted, or if u and v are no longer adjacent, discard the candidate as stale and continue extracting.
3. Output this pair.
4. Delete u and v from the linked list.
5. If the predecessor p of u and successor q of v both exist, insert the new adjacent candidate (p, q) into the priority queue.

If there are still ties after both difference and sum are equal, any deterministic tie-breaking rule, for example by original indices, may be used.

Correctness. For any finite sorted set, a pair of minimum absolute difference must be adjacent in sorted order: if $x < y < z$, then $z - x > (y - x)$ and $z - x > (z - y)$, so the non-adjacent pair (x, z) cannot be uniquely better than its adjacent subpairs. Hence at every step the pair chosen by the process is among adjacent linked-list pairs. The priority queue contains all current adjacent pairs, plus stale pairs from earlier states. Stale pairs are explicitly discarded. Therefore the first valid pair extracted from the priority queue is exactly the adjacent pair with minimum difference and, among those, maximum sum. Deleting it and inserting only the new outer-neighbor pair restores the set of adjacent candidates, because no other adjacencies change. By induction over the n removal steps, the algorithm outputs exactly the required removal order.

Running time. Sorting costs $O(n \log n)$ for $2n$ numbers. Initially $O(n)$ adjacent candidates are inserted, and each deletion step inserts at most one new candidate. Thus there are $O(n)$ heap insertions and extractions, each costing $O(\log n)$. The total running time is $O(n \log n)$ and the memory is $O(n)$.

Problem 12. Priority queue functionality using sorted decreasing arrays

Suppose we do not want to implement a binary heap. Show how to implement the same priority-queue functionality using arrays sorted in decreasing order. What array length should be chosen, and what are the asymptotic operation bounds?

Solution.

Idea. Use square-root decomposition. Split the elements into blocks, each stored as an array sorted in decreasing order. Then the minimum of a block is at its right end. Insertions cost the length of one block; finding the global minimum costs the number of blocks. Balance these two quantities.

Algorithm. Assume an upper bound N on the number of simultaneously stored elements or on the number of operations; otherwise use standard global rebuilding when the size changes by a constant factor. Choose block length

$$B = \lceil \sqrt{N} \rceil.$$

Maintain a list of blocks, each of size at most B , sorted in decreasing order. The operations are:

- **INSERT(x):** insert x into the last block, shifting elements inside that block to keep it sorted decreasing. If the last block already has size B , create a new block. Cost $O(B)$.
- **GETMIN:** scan the last element of every nonempty block and return the smallest among them. Cost $O(\text{\#blocks}) = O(N/B)$.
- **EXTRACTMIN:** first find the block containing the global minimum as in GETMIN; since blocks are decreasing, that minimum is the last element of its block. Remove it. If a block becomes empty, delete the block. Cost $O(N/B)$.

For a max-priority queue, store blocks in increasing order or scan the left endpoints instead.

Correctness. Each block is sorted decreasing, so its minimum is exactly its last element. Therefore the global minimum over all stored elements is the minimum among the block endpoints inspected by GETMIN. The insertion procedure places the new element into exactly one block

and restores that block's sorted order, so the block invariant is preserved. `EXTRACTMIN` removes exactly the endpoint identified as the global minimum, and deleting an empty block does not affect the represented set. Hence the data structure implements the priority-queue operations correctly.

Running time. The costs are $O(B)$ for insertion and $O(N/B)$ for `GETMIN` and `EXTRACTMIN`. Choosing $B = \Theta(\sqrt{N})$ balances the terms and gives $O(\sqrt{N})$ worst-case time per operation, with $O(N)$ memory. With global rebuilding, the same bounds hold with N replaced by the current scale of the data structure.